

Lab 1 - ELA408

Kim Svedberg



I. INTRODUCTION

This paper presents the design and results for Lab 1 [1]. The system implemented is of a differential drive model inside of a unicycle model with four different behaviors: Go-To-Goal (GTG), Avoid Obstacle (AO), and Follow Wall (FW) clockwise (c) or counter-clockwise (cc).

Differential drive model is a control model for robots with two motorized wheels, see Figure 1 for an example. The differential drive model works by controlling the velocities of the left (v_l) and right (v_r) wheel to decide the speed and direction of the robot, however, this is not ideal as it is unintuitive to control the robot this way [2]. We would instead like to control the robot using only linear and angular velocity, as this would simplify the control. For the differential drive model, with wheel velocities v_l and v_r , wheel radius r and wheel base l , the $\dot{x}$, $\dot{y}$ and $\dot{\theta}$ can be calculated by:

$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_l) \cdot \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_l) \cdot \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_l) \end{cases} \quad (1)$$

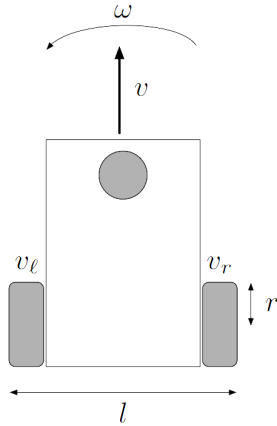


Fig. 1. Example of a differential drive robot with two motorized wheels (v_l) and (v_r), wheel radius r and wheel base l . The front wheel is used for maintaining stability.

To work with the desired linear and angular velocity, the unicycle model can be used together with the differential drive model. The unicycle model is much easier to control and model, only requiring specifying the desired velocity and direction rather than individual wheel velocities. For the unicycle model, with linear velocity v and angular velocity ω , the $\dot{x}$, $\dot{y}$ and $\dot{\theta}$ can be calculated by:

$$\begin{cases} \dot{x} = v \cdot \cos \theta \\ \dot{y} = v \cdot \sin \theta \\ \dot{\theta} = \omega \end{cases} \quad (2)$$

By equating equation (1) and (2), we get the following relation for the wheel velocity v_l and v_r :

$$v_l = \frac{2v - \omega l}{2r} \quad (3)$$

$$v_r = \frac{2v + \omega l}{2r} \quad (4)$$

By using equation (3) and (4) in (1), we can control a differential drive robot simply by specifying the desired linear v and angular ω velocity.

II. METHOD

The system was designed and simulated in Matlab R2025b, Simulink and Stateflow using the provided model and maze for Lab 1 ("SimulationWithSensor.zip"). The physical and control parameters used in the simulation can be seen in Table I.

TABLE I
PARAMETERS USED IN THE SIMULATION.

Parameters	Values
Wheel radius (r)	0.1
Wheel base (l)	0.5
Follow wall threshold (Δ_1)	2
Avoid obstacle threshold (Δ_2)	0.5
Constant velocity (v)	0.5
Proportional gain (P)	10
Goal tolerance (tol)	0.1

A. Behavior Design

There are a total of four different behaviors/states to the system: Go-to-Goal (GTG), Avoid Obstacle (AO), and Follow Wall (FW) in both clockwise (c) and counter-clockwise (cc) directions. The GTG behavior was already provided by the assignment beforehand so only the AO and FW state and transitions were needed to be implemented [1]. The state and transitions was implemented using a Stateflow finite state machine, see Figure 2 for the state machine logic.

In the GTG state, the robot calculates a direct trajectory toward the goal position. Upon detecting an obstacle within the sensor's range, the system transitions to the FW state. The selection between the clockwise ($\mathbf{u}_c$) and counter-clockwise ($\mathbf{u}_{cc}$) directions is determined by evaluating the dot product of each tangential vector relative to the $\mathbf{u}_{GTG}$ vector. The AO state serves as a safety state, triggered only when an obstacle is within the avoidance threshold (Δ_2). This state moves the

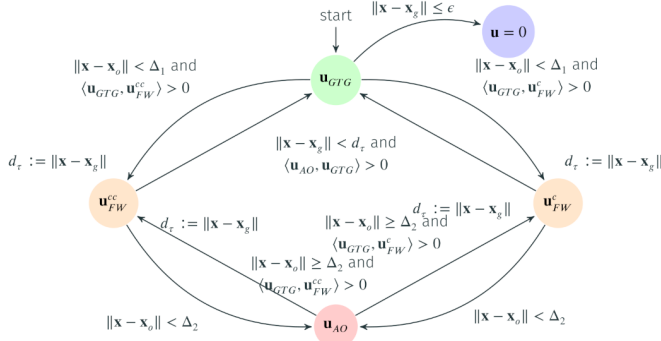


Fig. 2. Provided state machine suggested and implemented for the behavior transitions [2].

robot away from the obstacle to maintain a safe distance before resuming FW or GTG state.

To calculate the distance between two positions $\|X - X_x\|$, the Matlab *norm* function was used, which calculates the Euclidean distance between two points. This was used to calculate the distance between the robot position (X) and targets (goal X_g or obstacle X_o position). All of the vectors ($\mathbf{u}_{GTG}$ and $\mathbf{u}_{AO}$) are calculated and normalized using following formula:

$$\mathbf{u}_{GTG} = \frac{X_g - X}{\text{norm}(X_g - X) + \epsilon} \quad (5)$$

$$\mathbf{u}_{AO} = \frac{X - X_o}{\text{norm}(X - X_o) + \epsilon} \quad (6)$$

where ϵ is a small constant (2.2×10^{-16}) included to prevent division by zero, $\mathbf{u}_{GTG}$ is the unit vector directed toward the goal, and $\mathbf{u}_{AO}$ is the unit vector directed away from the obstacle.

To calculate the angle between two vectors, the Matlab *dot* function was used to compute the scalar product between two vectors $\mathbf{u}_a$ and $\mathbf{u}_b$: $\langle \mathbf{u}_a, \mathbf{u}_b \rangle$.

These calculations are important for the transition within the Stateflow machine. For example, the transition from GTG state to the FW state occurs when the distance to an obstacle $\|X - X_o\|$ falls below the threshold Δ_1 , while the choice between clockwise and counter-clockwise wall following is determined by which tangential vector ($\mathbf{u}_c$ or $\mathbf{u}_{cc}$, respectively) aligns more closely with the goal direction $\mathbf{u}_{GTG}$.

B. Sensor Processing

The sensors provided by the assignment gives a 5×5 grid surrounding the robot, allowing it to detect obstacles within a two space radius in all directions [1]. This grid is aligned with the global frame, meaning it does not rotate with the robot. We therefore treat the 5×5 grid as a local coordinate system where the robot is always positioned at the center (3, 3).

To identify obstacles, we look for cells containing a value of 0. If multiple obstacles are detected, the system calculates the distance from the robot's center (3, 3) to every detected obstacle cell. The nearest obstacle cell to the robot is then selected to calculate the obstacle position X_o (robot's position

+ distance to obstacle) and the avoidance vector $\mathbf{u}_{AO}$ (eq. 6), which can then be used to calculate $\mathbf{u}_c$ and $\mathbf{u}_{cc}$:

$$\mathbf{u}_c = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \cdot \mathbf{u}_{AO} \quad (7)$$

$$\mathbf{u}_{cc} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \cdot \mathbf{u}_{AO} \quad (8)$$

C. Control and Error

The output of the planner is a desired velocity vector $\mathbf{v} = [v_x, v_y]^T$. To get the desired heading θ_d the Matlab *atan2* function was used:

$$\theta_d = \text{atan2}(v_y, v_x) \quad (9)$$

The angular error e is defined as the difference between the desired heading θ_d and the current measured heading θ of the robot. To ensure the robot always turns along the shortest angular path, we make $e \in [-\pi, \pi]$ using the *atan2* function, the error is normalized using the following relation:

$$e = \theta_d - \theta \quad (10)$$

$$e_{norm} = \text{atan2}(\sin e, \cos e) \quad (11)$$

The resulting error is then used to calculate the angular velocity ω by the proportional control law $\omega = P \cdot e_{norm}$. Finally, ω can be used to calculate v_l and v_r as in equation 3 and 4 to control the robot (velocity v is constant).

III. RESULTS

The robot path through the maze can be seen in Figure 3. The path taken is similar to the expected path according to the instructions provided by Lab 1 instructions [1].

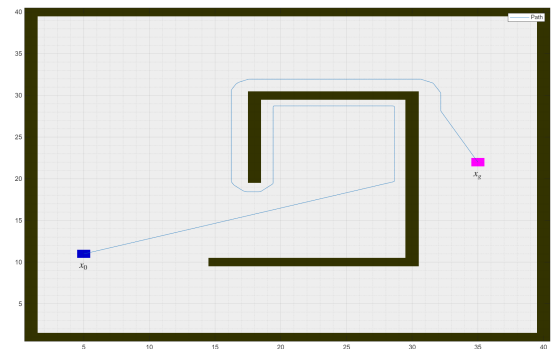


Fig. 3. The robot path through the maze. Completes the whole maze without touching the walls once.

Figure 4 shows the position (x and y) and heading/angle (θ and θ_d) of the robot over time. The robot completes the maze within 190 seconds. All of the plots shows straight lines with sharp corners, indication fast and sharp turning angles.

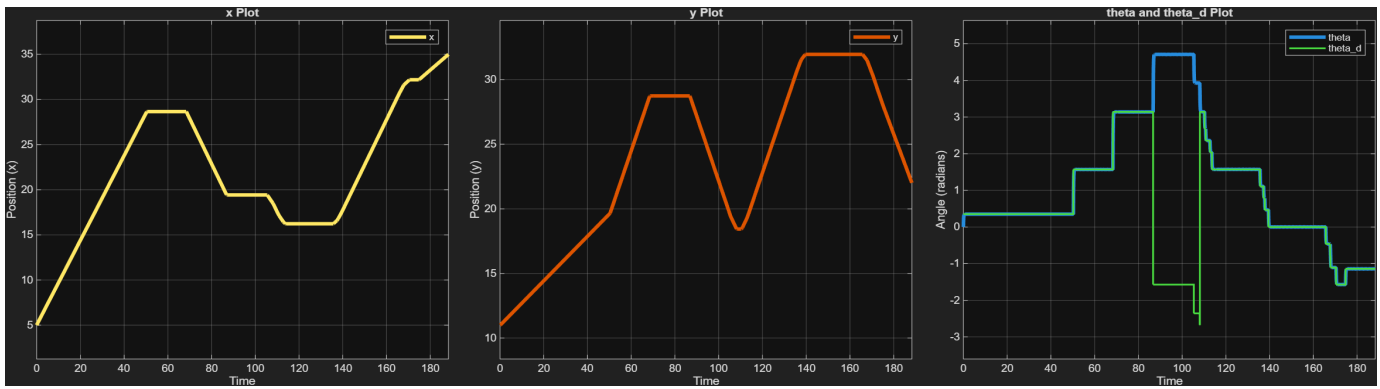


Fig. 4. Plot of x (yellow line), y (orange line), θ (blue line), and θ_d (green line, ref.) of the robot over time. θ and θ_d appears to differ in the middle because we wrap the error between $[-\pi, \pi]$, but it is the same angle.

IV. DISCUSSION

Implementing this system into a physical platform would introduces several challenges that are absent in the Simulink environment. Currently, the robot calculates its position by integrating $\dot{x}$ and $\dot{y}$. However, real robots faces odometry drift caused by wheel slip or uneven surface contact, meaning its internal pose estimate will slowly deviate from reality unless corrected by wheel encoders or other sensors. Furthermore, the P-controller currently uses a constant gain of 10 to command angular velocity, but physical motors are restricted by RPM and torque limits, so tuning to the controller may be required depending on the motors used. Finally, unlike the instantaneous calculations of the simulation, hardware microcontrollers also introduce latency. If the robot's velocity v is too high, processing delays could lead to collisions before the follow wall behavior can execute.

In the Simulink model, the sensor provides a perfect 5x5 matrix of the environment without any noise or delays, which is not realistic. Some of the sensors that could be used are:

- **Ultrasonic sensor:** Measures distance by emitting sound waves. Cheap but different surfaces/materials reflects sounds differently.
- **Infrared (IR) sensor:** Detects infrared radiation emitted or reflected by objects. Good for short range detection, but their accuracy depends heavily on the color and reflectivity of the obstacle.
- **Light Detection and Ranging (LiDAR) sensor:** Measures distance by emitting light. More advanced option, but different surfaces reflect light differently.

Real sensors often also produces noise and not stable readings. To mitigate this, some filtering would be required.

- **Median filter:** Taking the middle value of the last n reading to ignore sudden spikes. (n is the window size)
- **Low-pass filter:** Smoothing the sensor data over time.
- **Kalman filter:** A more advanced approach that estimates the robot's state by weighing the sensor data against the robot's predicted movement.

Finally, behavior based robots, which this system is, relies on reactive logic rather than global path planning. While the robot successfully navigated through the maze, the path is inefficient because it only makes decisions based on the

two space radius it can currently "see". In more complex environments, this can lead to a sub-optimal path or prevent the robot to ever find a path to the goal. Additionally, the lack of a global map or predictive obstacle avoidance may result in collisions with moving objects in more dynamic environments.

REFERENCES

- [1] MDU - ELA408, "Lab 1 - behavioral robotics," https://canvas.mdu.se/courses/18688/assignments/116234?module_item_id=645512, accessed 11-04-2026.
- [2] L. C. Eriksson, "Lecture 2 - kinematics and behavioral robotics," https://canvas.mdu.se/courses/18688/files/2895045?module_item_id=677798, 2026, accessed 11-04-2026.